Reinforcement Learning

Master Summary

Stefan Eder

Based on Sutton & Barto (2nd ed.) and lecture notes

This document is a compact summary of the Reinforcement Learning course, covering finite MDPs, bandits, dynamic programming, Monte Carlo methods, temporal-difference learning, and policy gradient methods. It is designed as a study aid alongside the textbook and lecture notes, not as a replacement for them.

If you spot an error or a gap, please open an issue or pull request at **github.com/stevetgit** — corrections are always welcome.

Good study materials should be shared, not gatekept. If this helped you, take it as a reason to help the next colleague you see struggling.

March 13, 2026

Contents

1 Finite Markov Decision Processes . . . **1**
- 1.1 The agent–environment interface . . . 1
- 1.2 Markov property and MDP dynamics . . . 2
- 1.3 Goals, rewards, and returns . . . 2
- 1.4 Value functions . . . 3
- 1.5 Optimal value functions and Bellman optimality . . . 4
- 1.6 Exam questions . . . 4

2 Multi-Armed Bandits . . . **6**
- 2.1 Problem setup . . . 6
- 2.2 Exploration vs. exploitation . . . 6
- 2.3 Action-value methods . . . 6
- 2.4 ε-greedy . . . 7
- 2.5 Optimistic initial values . . . 8
- 2.6 Upper Confidence Bound (UCB) . . . 8
- 2.7 Non-stationary problems . . . 9
- 2.8 Gradient bandits . . . 9
- 2.9 Exam questions . . . 10

3 Dynamic Programming . . . **11**
- 3.1 Iterative policy evaluation . . . 11
- 3.2 Policy improvement . . . 12
- 3.3 Policy iteration . . . 12
- 3.4 Value iteration . . . 13
- 3.5 Asynchronous DP . . . 13
- 3.6 Exam questions . . . 13

4 Monte Carlo Methods . . . **15**
- 4.1 MC prediction . . . 15
- 4.2 MC control . . . 16
  - 4.2.1 Exploring starts . . . 16
  - 4.2.2 On-policy MC control for ε-soft policies . . . 16
- 4.3 Off-policy MC and importance sampling . . . 17
- 4.4 Exam questions . . . 18

5 Temporal Difference Learning . . . **19**
- 5.1 TD(0) prediction . . . 19
- 5.2 TD control . . . 20
  - 5.2.1 SARSA (on-policy) . . . 20
  - 5.2.2 Off-policy SARSA and importance sampling . . . 21
  - 5.2.3 Expected SARSA . . . 21
  - 5.2.4 Q-learning (off-policy) . . . 21
- 5.3 n-step TD and the TD–MC spectrum . . . 22

5.4 Eligibility traces and TD(λ) 22
5.5 Exam questions 23

6 Policy Gradient Methods 25
6.1 The policy gradient objective 25
6.2 REINFORCE 25
6.3 Baselines and variance reduction 26
6.4 Actor–critic 26
6.4.1 PPO (brief) 27
6.5 Exam questions 27

1

Finite Markov Decision Processes

Reinforcement learning (RL) is the third major learning paradigm alongside supervised and unsupervised learning. In **supervised learning** we fit a mapping $x \mapsto y$ from labeled pairs; in **unsupervised learning** we discover structure in unlabeled data. RL is different: an **agent** interacts with an **environment** and learns to act so as to maximize cumulative reward. The environment is not a fixed dataset — it responds to the agent's actions, which makes the exploration–exploitation trade-off central to RL in a way that simply does not arise in the other two paradigms.

1.1 The agent–environment interface

At each discrete time step $t = 0, 1, 2, \ldots$ the agent observes the current state, chooses an action, and the environment returns a reward and a new state.

Trajectory

The agent observes state $S_t \in \mathcal{S}$ and selects action $A_t \in \mathcal{A}(S_t)$. The environment responds with reward $R_{t+1} \in \mathbb{R}$ and next state S_{t+1}. The resulting sequence

$$S_0,\ A_0,\ R_1,\ S_1,\ A_1,\ R_2,\ \ldots$$

is called a *trajectory* (or history).

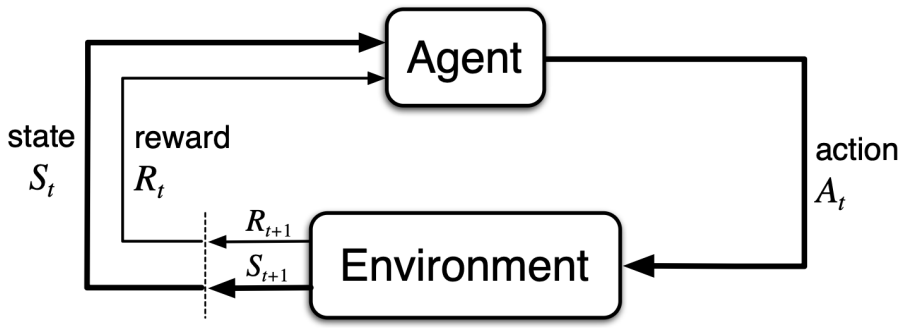

Figure 1.1: Standard agent–environment interaction at time t.

The agent's behavior is encoded in a *policy*.

Policy

A *policy* π specifies how the agent acts. In the most general (stochastic) case,

$$\pi(a \mid s) \;=\; \Pr(A_t = a \mid S_t = s).$$

A policy is *deterministic* if it selects a single action with probability 1 in each state.

1.2 Markov property and MDP dynamics

The Markov property makes the state a sufficient statistic for future behavior: knowing the present state is enough; the history adds nothing.

Markov Property

A state signal is *Markov* if the next-state and reward distribution depends on the past only through the current state and action:

$$\Pr(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a,\ \text{past}) \;=\; \Pr(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a).$$

A classic illustration: if the state is the full chess board position, the next position depends only on that board and the move — Markov. If instead the state were only the knight's current square (ignoring the rest of the board), it would *not* be Markov, because the hidden pieces still influence what moves are legal.

A state space can be discrete (countably many states, e.g. chess positions) or continuous (uncountably many, e.g. real-valued sensor readings). The complete model of how the environment transitions is given by the *dynamics function*.

Finite MDP

A **finite Markov decision process (finite MDP)** is defined by finite state and action sets together with the *dynamics*

$$p(s', r \mid s, a) \;=\; \Pr(S_{t+1} = s',\ R_{t+1} = r \mid S_t = s,\ A_t = a).$$

Derived quantities used throughout:

$$r(s, a) = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a], \qquad p(s' \mid s, a) = \Pr(S_{t+1} = s' \mid S_t = s, A_t = a).$$

Intuitively, p is the environment's rulebook: "if you do a in s, what tends to happen next?"

1.3 Goals, rewards, and returns

The **reward hypothesis** states that all goals can be formalized as maximizing a cumulative scalar reward signal. The reward R_{t+1} is the immediate feedback; what the agent actually cares about is the long-run sum. Importantly, reward is a *design choice*, not synonymous with "winning": for chess one might use $+1$ win, 0 draw, -1 loss; for navigation, -1 per time step and 0 at the goal encourages the shortest path; for robotics, shaping the reward incorrectly can lead to unintended behavior.

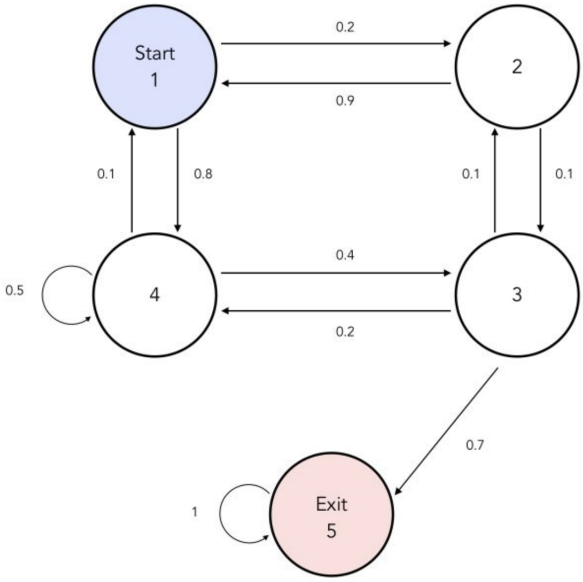

Figure 1.2: A small MDP visualized as a directed graph with action-labeled transitions.

Return and Discount Factor

For an **episodic task** that terminates at time T, the **discounted return** from time t is

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1},$$

where $\gamma \in [0, 1]$ is the **discount factor**. For a **continuing task** (no terminal time), discounting with $\gamma < 1$ keeps the infinite sum finite and expresses preference for sooner rewards.

A key identity simplifies computing returns backward through an episode:

$$G_t = R_{t+1} + \gamma G_{t+1}.$$

Starting from $G_T = 0$ and iterating back is far more efficient than summing forward from each t. As a sanity check: for a continuing task with constant reward $R_{t+1} = r$ and $\gamma \in [0, 1)$, the geometric series gives $G_t = r/(1 - \gamma)$.

1.4 Value functions

Value functions answer the question: "how good is it to be in state s (or to take action a in s) under a given policy?" They are the central objects computed and approximated throughout RL.

State-Value and Action-Value Functions

Given a policy π, the **state-value function** and **action-value function** are

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s], \qquad q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a].$$

Intuitively: $v_\pi(s)$ is how good it is to be in s under π; $q_\pi(s, a)$ is how good it is to take a in s and then follow π.

Both value functions satisfy recursive *Bellman equations* that express current value as one-step reward plus discounted future value.

Bellman Expectation Equations

For a finite MDP under policy π:

$$v_\pi(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma\, v_\pi(s')\big],$$

$$q_\pi(s,a) = \sum_{s',r} p(s', r \mid s, a)\Big[r + \gamma \underbrace{\sum_{a'} \pi(a' \mid s')\, q_\pi(s', a')}_{= v_\pi(s')}\Big].$$

Reading the first equation: "value equals expected immediate reward plus discounted value of the next state."

1.5 Optimal value functions and Bellman optimality

A policy π is *optimal* if $v_\pi(s) \geq v_{\pi'}(s)$ for all s and all policies π'. Such a policy always exists for finite MDPs.

Optimal Value Functions

$$v_*(s) = \max_\pi v_\pi(s), \qquad q_*(s,a) = \max_\pi q_\pi(s,a).$$

An optimal policy π_* attains these maxima simultaneously for all s.

The optimal value functions are the unique solutions to the Bellman *optimality* equations, which replace the policy-weighted sum by a *max*:

Bellman Optimality Equations

$$v_*(s) = \max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma\, v_*(s')\big],$$

$$q_*(s,a) = \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma \max_{a'} q_*(s', a')\big].$$

A greedy policy w.r.t. q_* is optimal: $\pi_*(s) \in \arg\max_a q_*(s,a)$.

1.6 Exam questions

Exam: Matching Concepts

Q. Match each description to the correct concept: (A) defines the agent's behavior at a given time, (B) mimics the behavior of the environment, (C) immediate scalar feedback provided each step, (D) expected sum of future rewards indicating long-term desirability. Concepts: (1) policy, (2) model, (3) reward signal, (4) value function.

A. A $\to$ policy, B $\to$ model, C $\to$ reward signal, D $\to$ value function.

Exam: Return Computation

Q. Let $\gamma = \frac{1}{2}$ and rewards $R_1 = -7$, $R_2 = 6$, $R_3 = 1$, $R_4 = 5$, $R_5 = 2$ with terminal time $T = 5$. Compute $G_0, \ldots, G_5$.

A. Work backward using $G_t = R_{t+1} + \gamma G_{t+1}$ and $G_5 = 0$:

$$G_4 = 2, \quad G_3 = 5 + \tfrac{1}{2} \cdot 2 = 6, \quad G_2 = 1 + \tfrac{1}{2} \cdot 6 = 4,$$

$$G_1 = 6 + \tfrac{1}{2} \cdot 4 = 8, \quad G_0 = -7 + \tfrac{1}{2} \cdot 8 = -3.$$

Exam: Undiscounted Return for Non-Episodic Tasks

Q. "It is better to use the undiscounted return ($\gamma = 1$) for non-episodic tasks." True or false?

A. False in general. For continuing tasks, an undiscounted infinite sum can diverge. Typical fixes: $\gamma < 1$ (bounded rewards), or an average-reward objective.

2

Multi-Armed Bandits

The multi-armed bandit is the simplest setting where the exploration–exploitation trade-off appears in pure form. There is no state transition: the agent simply picks an arm at each step and observes a reward. Despite this simplicity, bandits introduce every key idea needed for the full RL setting.

2.1 Problem setup

k-Armed Bandit

A **k-armed bandit** has k actions (arms) $a \in \{1, \ldots, k\}$. At each time step t the agent selects A_t and observes reward R_t. The **true value** of arm a (stationary case) is

$$q_*(a) \;=\; \mathbb{E}[R_t \mid A_t = a].$$

The agent maintains an **estimate** $Q_t(a)$ and a **count** $N_t(a)$ = number of times arm a has been selected up to time t.

A problem is **stationary** if $q_*(a)$ is constant over time; **non-stationary** if the reward distributions drift. This distinction drives the choice of update rule later.

2.2 Exploration vs. exploitation

Exploitation selects the arm with the highest current estimate (greedy w.r.t. Q_t), extracting maximum immediate reward given current knowledge. **Exploration** instead tries other arms to improve estimates, accepting lower short-term reward for potentially better long-term performance. You cannot do both simultaneously; the trade-off is the central problem of bandits.

2.3 Action-value methods

The sample-average estimate is the natural unbiased estimator:

Sample-Average Estimate

If arm a has been selected $N_t(a) \geq 1$ times:

$$Q_t(a) = \frac{1}{N_t(a)} \sum_{i=1}^{N_t(a)} R^{(i)}(a).$$

Computing this from scratch each step is wasteful. The incremental form processes one new observation at a time:

Incremental Update (ISAM)

After selecting A_t and observing R_t, update only $Q(A_t)$:

$$Q_{t+1}(A_t) = Q_t(A_t) + \frac{1}{N_{t+1}(A_t)}\big(R_t - Q_t(A_t)\big).$$

More generally, with a constant step size $\alpha \in (0, 1]$:

$$Q_{t+1}(A_t) = Q_t(A_t) + \alpha\big(R_t - Q_t(A_t)\big).$$

The constant-α version is an **exponentially recency-weighted** average — ideal for non-stationary problems where older rewards should count less.

Incremental Update Calculation

If $Q_t(a) = -3$, $R_t = 2$, $\alpha = 0.1$:

$$Q_{t+1}(a) = -3 + 0.1\big(2 - (-3)\big) = -3 + 0.5 = -2.5.$$

2.4 ε-greedy

ε-Greedy Action Selection

With probability $1 - \varepsilon$ choose the greedy action $A_t \in \arg\max_a Q_t(a)$, and with probability ε choose uniformly at random from $\{1, \ldots, k\}$.

With constant $\varepsilon > 0$, exploration never stops: a fraction ε of steps will forever be random. Smaller ε typically yields higher asymptotic reward but slower early learning. Pure greedy ($\varepsilon = 0$) can get stuck on a suboptimal arm because early noise drives it to commit before all arms are tried. **GLIE** schedules (Greedy in the Limit with Infinite Exploration, e.g. $\varepsilon_k = 1/k$) decay ε to zero while still exploring infinitely often — a condition required for convergence guarantees.

Greedy Selection Probability

Under ε-greedy with uniform exploration, the probability of selecting the greedy action is

$$\Pr(\text{greedy}) = (1 - \varepsilon) + \frac{\varepsilon}{k}.$$

For $k = 10$, $\varepsilon = 0.7$: $\Pr(\text{greedy}) = 0.3 + 0.07 = 0.37$.

Note: if a *non-greedy* action was selected, the ε-branch definitely fired. If the greedy action was selected, it could have come from either branch.

2.5 Optimistic initial values

Optimistic Initialization

Set $Q_1(a)$ to values *higher* than any plausible reward (requiring rough knowledge of the reward scale). Under pure greedy selection, untried arms keep their optimistic estimates and will therefore be selected, driving early exploration without any randomness.

Works well in **stationary** environments. In non-stationary settings the initial optimism is "washed out" once values converge: if rewards then shift, the mechanism provides no further exploration.

Exam Nuances: Exploration Hyperparameters

- **ε-greedy** controls exploration explicitly via ε: larger $\varepsilon \Rightarrow$ more random steps.
- **UCB** controls exploration via c: larger $c \Rightarrow$ larger uncertainty bonus.
- **Optimistic initialization** requires a reward-scale prior (an implicit, one-time exploration parameter), but does not expose a run-time knob.
- **Gradient bandits** have no explicit exploration rate. They explore because they *sample* from a softmax. The step size α controls learning speed, not exploration probability.

If rewards are strictly negative, $Q_1(a) = 0$ is already optimistic.

2.6 Upper Confidence Bound (UCB)

UCB Action Selection

$$A_t \in \arg\max_a \left[Q_t(a) + c\sqrt{\frac{\ln t}{N_t(a)}} \right],$$

where $c > 0$ controls exploration. Each action is selected at least once before the formula is applied, so $N_t(a) \geq 1$. Setting $c = 0$ reduces UCB to pure greedy.

Note that UCB naturally revisits neglected arms: if arm a is skipped for a while, $N_t(a)$ stays constant while $\ln t$ grows, so the bonus slowly *increases* until the arm becomes attractive again.

Why the UCB Bonus Works

The bonus $c\sqrt{\ln t / N_t(a)}$ is large when arm a is uncertain (small $N_t(a)$), making UCB "optimistic in the face of uncertainty." As a is sampled more, $N_t(a)$ grows and the bonus shrinks — once you are confident an arm is bad, it stops looking attractive.

UCB therefore explores *selectively*, prioritizing arms where evidence is weak and a surprise could still happen. In contrast, ε-greedy explores uniformly at random regardless of confidence. The $\ln t$ numerator slowly increases pressure to re-examine arms, but the growth is dominated by $N_t(a)$, so focus naturally shifts from exploration to exploitation over time.

In short: **explore what you are unsure about; exploit what you are confident is**

good.

The classic UCB derivation assumes **stationary** bandits. In strongly non-stationary settings, modifications such as sliding-window or discounted UCB are needed.

2.7 Non-stationary problems

When $q_*(a)$ drifts over time, the sample average gives equal weight to all past observations — including stale ones. The constant step-size rule $Q_{t+1}(A_t) = Q_t(A_t) + \alpha(R_t - Q_t(A_t))$ with fixed α exponentially downweights older rewards, letting the estimate track changes. This comes at the cost of never fully converging even in stationary settings.

2.8 Gradient bandits

Preference Parameters

Instead of estimating action values, gradient bandits maintain a real-valued **preference** (logit) $H_t(a) \in \mathbb{R}$ for each arm. Only *differences* matter: adding a constant to all $H_t(a)$ leaves the policy unchanged. Large $H_t(a)$ means the policy is more likely to pick a.

Preferences are converted to action probabilities by the softmax:

Softmax Policy and Gradient Update

$$\pi_t(a) = \frac{\exp(H_t(a))}{\sum_b \exp(H_t(b))}.$$

Actions are sampled from $\pi_t(\cdot)$ (not greedy). After observing R_t with baseline $\bar{R}_t$ (often the running average reward):

$$H_{t+1}(a) = \begin{cases} H_t(a) + \alpha(R_t - \bar{R}_t)(1 - \pi_t(a)) & a = A_t, \\ H_t(a) - \alpha(R_t - \bar{R}_t)\,\pi_t(a) & a \neq A_t. \end{cases}$$

What the Gradient Update Is Doing

Think of π_t as a weighted lottery. After picking A_t:

- If $R_t > \bar{R}_t$ (better than average): increase $H(A_t)$, making A_t more likely; decrease others slightly.
- If $R_t < \bar{R}_t$ (worse than average): decrease $H(A_t)$; increase others slightly.

The factors $(1 - \pi_t(A_t))$ and $\pi_t(a)$ scale the update: large adjustments when the action was unlikely (surprising), small adjustments when it was already highly probable. This is gradient ascent on expected reward w.r.t. the preference parameters.

Key Distinctions

Preferences $\neq$ value estimates: $H_t(a)$ defines π_t via softmax; it is *not* an estimate of $Q(a)$.

Baseline must be action-independent: using $Q_t(A_t)$ as a baseline is generally invalid. A valid action-independent baseline is $\sum_a \pi_t(a)Q_t(a)$ (the expected value under the current policy).

Exam wording trap: "actions are selected proportional to their preference" is technically false ($\pi_t(a) \propto \exp(H_t(a))$, not $\propto H_t(a)$). Past exams have marked this **True**, treating "preference" loosely as the softmax sense.

2.9 Exam questions

Exam: Optimistic Initialization

Q. "Optimistic initial values bias the agent toward exploring actions with higher estimated rewards." True or false?

A. True. Untried arms keep optimistic estimates; greedy selection is therefore drawn to them first.

Exam: UCB

Q. "UCB balances exploration and exploitation by adding an uncertainty term to each action value." True or false?

A. True. The defining idea is $\arg\max_a \left[Q_t(a) + \text{bonus}(a)\right]$.

Exam: Gradient Bandits

Q. "In gradient bandits, actions are selected proportional to their action preference." True or false?

A. True (as marked in past exams, using "preference" in the softmax sense: $\pi_t(a) \propto \exp(H_t(a))$). Strictly, proportional to the *exponentiated* preference.

3

Dynamic Programming

Dynamic programming (DP) in RL refers to a family of *planning* algorithms: they compute value functions and optimal policies using a **known MDP model** $p(s', r \mid s, a)$. Because the model is available, DP performs **expected backups** (sums over all possible next states) rather than learning from sampled experience. This makes DP exact but inapplicable whenever the model is unknown.

Three terms appear throughout: a **backup** updates one value $V(s)$ using the Bellman equation; a **sweep** is one pass that performs at most one backup per state $s \in \mathcal{S}$; **iterative evaluation** repeats sweeps until convergence.

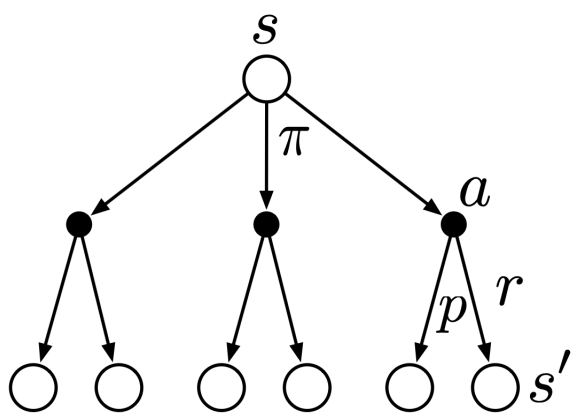

Figure 3.1: DP backup: take the expectation over all immediate successors (requires access to the model).

3.1 Iterative policy evaluation

Given a fixed policy π, we want to compute v_π exactly. The Bellman expectation equation gives a fixed-point condition: $v_\pi = T^\pi v_\pi$, where the Bellman expectation *operator* T^π is:

Bellman Expectation Operator

$$(T^\pi v)(s) \;=\; \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma\, v(s')\big].$$

The true value function v_π is the unique fixed point satisfying $T^\pi v_\pi = v_\pi$.

Repeated application of T^π converges to v_π from any initialization:

Iterative Policy Evaluation (Synchronous)

Initialize V_0 arbitrarily and iterate:

$$V_{k+1}(s) \;=\; (T^\pi V_k)(s) \;=\; \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma\, V_k(s')\big].$$

Stop when $\max_s |V_{k+1}(s) - V_k(s)| < \theta$.

In-Place (One-Array) Evaluation

Update $V(s) \leftarrow (T^\pi V)(s)$ immediately, using the most recent available values for successor states. Convergence is typically *faster* (Gauss–Seidel style), but the update order can affect intermediate estimates. Unlike the synchronous version, later states in the same sweep may see already-updated values from earlier in that sweep.

Iterative policy evaluation is a **bootstrapping** method: it updates $V(s)$ using current estimates $V(s')$ of successor states.

3.2 Policy improvement

After evaluating v_π, we can compute the action-value function and look for a better policy:

Policy Improvement

Given v_π, compute

$$q_\pi(s, a) \;=\; \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma\, v_\pi(s')\big],$$

and form the greedy policy $\pi'(s) \in \arg\max_a q_\pi(s, a)$.

Policy improvement theorem: $v_{\pi'}(s) \geq v_\pi(s)$ for all s. The improvement may be strict (genuinely better) or non-strict ($\pi' = \pi$ already, meaning π was optimal).

3.3 Policy iteration

Policy iteration alternates between evaluation and improvement until the policy stabilizes:

1. **Evaluate** the current π: run iterative evaluation to obtain v_π (approximately).
2. **Improve**: set $\pi \leftarrow \text{Greedy}(q_\pi)$.
3. Repeat until the policy is unchanged (*policy stable*).

For finite MDPs in the standard discounted setting, policy iteration terminates at an optimal policy after a finite number of improvements. **Modified policy iteration** truncates step 1 to just a few sweeps before improving, interpolating between policy iteration (full evaluation) and value iteration (one backup per sweep).

3.4 Value iteration

Instead of evaluating the current policy to convergence, value iteration applies the *optimality* operator T^* in each sweep:

Value Iteration

The Bellman optimality operator:

$$(T^*v)(s) \;=\; \max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma\, v(s')\big].$$

The fixed point $v_* = T^* v_*$ is the unique optimal value function.

Value iteration iterates $V_{k+1}(s) = (T^* V_k)(s)$ until convergence, then extracts a greedy optimal policy:

$$\pi_*(s) \in \arg\max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma\, V(s')\big].$$

Value Iteration as Policy Iteration with One Backup

Value iteration is equivalent to policy iteration where the evaluation step is truncated to a *single* backup. It implicitly evaluates the greedy policy w.r.t. the current estimate — without maintaining an explicit policy variable. Convergence is therefore typically asymptotic, not finite.

3.5 Asynchronous DP

Synchronous DP sweeps every state on each pass, which can be expensive. **Asynchronous DP** relaxes this: it updates states in any order, possibly skipping some between passes, applying either the evaluation backup $V(s) \leftarrow (T^\pi V)(s)$ or the optimality backup $V(s) \leftarrow (T^* V)(s)$. The extreme version is **prioritized sweeping**: always update the state with the largest Bellman residual, so effort is concentrated where estimates are furthest from being fixed points.

3.6 Exam questions

Exam: Optimal Policy via Linear Bellman Equations

Q. "Given an arbitrary policy, the optimal policy can be computed directly by solving the Bellman equations as a linear system." True or false?

A. False. The *linear* Bellman system solves v_π for a *fixed* π. Optimality requires a max (nonlinear condition).

Exam: Policy Evaluation Computation

Q. Policy evaluation with reward -4 per step, $\gamma = 0.5$, all initial values 0, terminal value 0. What is the value of state 0 after two sweeps?

A. Apply $V_{k+1}(s) = \sum_a \pi(a \mid s) \sum_{s'} p(s' \mid s, a)[-4 + 0.5\, V_k(s')]$. With $V_0 \equiv 0$: first sweep gives $V_1(s) = -4$ for nonterminal states. Second sweep (example: state 0 transitions to a nonterminal with prob. 0.5 and terminal with prob. 0.5): $V_2(0) = -4 + 0.5 \cdot (0.5 \cdot (-4) +$

$0.5 \cdot 0) = -5.$

Exam: Policy Improvement Computation

Q. With $R = -1$, $\gamma = 1$, compute $Q(\mathit{small})$ and $Q(\mathit{large})$ at state 0 and pick the greedy action.

A. $Q(a) = \sum_{s'} p(s' \mid 0, a)[-1 + V(s')]$.

$Q(\mathit{small}) = 0.5(-1+3)+0.5(-1-1) = 0, \quad Q(\mathit{large}) = 0.5(-1+5)+0.25(-1-3)+0.25(-1-7) = -1.$

Greedy choice: **small**.

Common Exam Pitfalls — DP

- Mixing up **evaluation** (compute v_π for fixed π) with **control** (improve π toward optimality).
- Forgetting DP is **model-based**: backups sum over all successors using $p(s', r \mid s, a)$.
- Confusing **two-array** (order irrelevant) with **in-place** (order matters) updates.
- "Greedy" in DP means $\arg\max$, not random exploration.
- Value iteration: convergence is **not finite**; use a threshold on the Bellman residual to decide when to stop.

4

Monte Carlo Methods

Monte Carlo methods bring model-free learning to the table: they estimate value functions from **sampled episodes** of actual (or simulated) experience, with no knowledge of $p(s', r \mid s, a)$ required. The key distinction from DP is that MC does **not bootstrap**: updates are based on observed returns G_t, not on estimates of successor values.

Monte Carlo Methods

MC methods are model-free and non-bootstrapping: they update value estimates using complete empirical returns from sampled episodes. MC typically assumes **episodic** tasks (or well-defined segments), because the return

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}$$

requires knowing the trajectory all the way to the terminal step T.

4.1 MC prediction

Given a fixed policy π, we estimate $v_\pi(s)$ by averaging observed returns across episodes. The two main variants differ in which visits to count.

First-Visit and Every-Visit MC Prediction

First-visit MC: for each episode, record the return G_t following the *first* visit to state s, then average across episodes: $V(s) \leftarrow$ mean of first-visit returns.

Every-visit MC: use the return following *every* visit to s in an episode.

First-visit returns across episodes are i.i.d. draws from the same distribution and converge by the law of large numbers. Every-visit returns within a single episode are generally *correlated* (multiple visits share the same trajectory), so they are not i.i.d., though every-visit MC still converges asymptotically.

The same idea applies to action values: treat each (s, a) pair as the "state" and accumulate first-visit (or every-visit) returns for $Q(s, a) \approx q_\pi(s, a)$.

Incremental Mean Update

If $V(s)$ is the current mean of n observed returns and a new return G arrives:

$$V(s) \leftarrow V(s) + \frac{1}{n+1}(G - V(s)).$$

Incremental Update

$V(s) = 4$ based on $n = 3$ returns; new return $G = 1$:

$$V(s) \leftarrow 4 + \tfrac{1}{4}(1 - 4) = 3.25.$$

4.2 MC control

For control, MC typically estimates **action values** $Q(s, a)$ rather than state values, because the greedy improvement step then becomes $\pi(s) \in \arg\max_a Q(s, a)$ — no model is needed.

4.2.1 Exploring starts

Greedy improvement poses a coverage problem: if the policy never takes action a in state s, $Q(s, a)$ never gets updated, and the policy might get stuck.

Exploring Starts (ES)

Assume every state–action pair (s, a) can start an episode with nonzero probability: $\Pr(S_0 = s, A_0 = a) > 0$ for all (s, a). This guarantees every pair is eventually tried, so a purely greedy policy still achieves coverage. (Primarily a theoretical device; often unrealistic in practice.)

Why Exploring Starts Are Needed

Without ES, a greedy policy that never tries some (s, a) will never correct $Q(s, a)$. If $Q(s, a)$ is initialized poorly, the policy is stuck. ES is a clean mathematical hack: by forcing random starts, every pair gets tried infinitely often in the limit, so Q eventually becomes accurate everywhere.

4.2.2 On-policy MC control for ε-soft policies

A more practical approach uses a policy that always explores a little:

ε-Soft Policy

π is **ε-soft** if $\pi(a \mid s) \geq \varepsilon/|\mathcal{A}(s)|$ for all (s, a), with $\varepsilon > 0$. No action is ever completely ruled out, guaranteeing ongoing exploration. The standard construction is ε-greedy w.r.t. Q.

Exploring Starts vs. ε-Soft

ES injects exploration only at episode *starts*: across many episodes, every (s, a) is tried as a starting pair. An ε-soft policy injects exploration *throughout* every episode: every time state s is visited, every action has nonzero probability. This prevents "never tried $\Rightarrow$ never learned" without requiring special episode initialization.

On-policy MC control repeats: (1) generate an episode under the current ε-soft policy, (2) update $Q(s, a)$ from observed returns (first-visit or every-visit), (3) improve π to be ε-greedy w.r.t. the updated Q. This is generalized policy iteration in the MC setting.

4.3 Off-policy MC and importance sampling

Off-Policy Learning

Learn about a **target policy** π using episodes generated by a different **behavior policy** μ. Required condition: **coverage**: $\mu(a \mid s) = 0 \Rightarrow \pi(a \mid s) = 0$ for all (s, a), so that importance sampling ratios are never undefined.

Why Importance Sampling Is Needed

Episodes from μ contain action choices that π would make with different probabilities. IS reweights each episode so that, in expectation, it counts as if generated by π. An episode where π is unlikely but μ is likely should count *less* (small weight), and vice versa.

Importance Sampling Ratio

For an episode segment t to T:

$$\rho_{t:T-1} = \prod_{k=t}^{T-1} \frac{\pi(A_k \mid S_k)}{\mu(A_k \mid S_k)}.$$

Then $\rho_{t:T-1} G_t$ is an unbiased estimate of $v_\pi(S_t)$ under μ.

Because the ratio is a product over many steps, it can become extremely large or small — a few rare episodes dominate the average. **Ordinary IS** is unbiased but potentially very high variance. **Weighted IS** normalizes by total weight, reducing variance at the cost of a small finite-sample bias:

Ordinary and Weighted Importance Sampling

Ordinary IS:

$$\hat{v}(s) = \frac{1}{n} \sum_{i=1}^{n} \rho^{(i)} G^{(i)}. \qquad \text{(unbiased, high variance)}$$

Weighted IS:

$$\hat{v}(s) = \frac{\sum_i \rho^{(i)} G^{(i)}}{\sum_i \rho^{(i)}}. \qquad \text{(biased for finite } n\text{, much lower variance)}$$

Weighted IS is the common practical default.

The weighted IS estimate can be maintained online via cumulative weights $C(s)$:

$$C(s) \leftarrow C(s) + \rho, \qquad V(s) \leftarrow V(s) + \frac{\rho}{C(s)}(G - V(s)).$$

4.4 Exam questions

Exam: Matching MC Methods

Q. Match: (A) $\varepsilon_k = 1/k$ for episode k; (B) greedy action with probability $1 - \varepsilon$ (constant ε); (C) learn about π from experience generated by μ. Methods: GLIE MC control; on-policy MC for ε-soft policies; off-policy learning.

A. A $\rightarrow$ GLIE MC control; B $\rightarrow$ on-policy ε-soft; C $\rightarrow$ off-policy.

Exam: MC Control and State Values

Q. "MC control methods compute an estimate of the state-value function v_π." True or false?

A. False (as stated). MC control is formulated via *action values* $Q(s, a)$; state values alone are insufficient for model-free greedy improvement.

Exam: Coverage Condition

Q. In off-policy MC prediction, what condition is needed for importance sampling to be well-defined?

A. Coverage: $\pi(a \mid s) > 0 \Rightarrow \mu(a \mid s) > 0$ for all (s, a) that can be visited; otherwise a ratio contains division by zero.

Common Exam Pitfalls — MC

- MC is **model-free** and **does not bootstrap**; it uses complete returns.
- First-visit vs every-visit: every-visit gives more samples per episode but within-episode samples are correlated.
- For control, MC uses $Q(s, a)$, not just $V(s)$.
- Off-policy: the coverage condition is $\mu = 0 \Rightarrow \pi = 0$ (not the other way around), and the ratio is a *product* over the full trajectory segment.
- Ordinary IS is unbiased but numerically unstable; weighted IS is the standard practical choice.

5

Temporal Difference Learning

Temporal-difference (TD) learning sits between DP and MC. Like MC it is **model-free**, learning from sampled experience. Like DP it **bootstraps**: it updates estimates using other learned estimates rather than waiting for a complete return. This combination — online, model-free, bootstrapping — makes TD the workhorse of practical RL.

A quick comparison of the three families:

- **DP:** model required; expected backups over all successors.
- **MC:** model-free; no bootstrapping; full episode needed.
- **TD:** model-free; bootstraps from one sampled transition.

5.1 TD(0) prediction

TD(0) Target and Error

For a transition $S_t \to S_{t+1}$ with reward R_{t+1}, the **TD(0) target** is $R_{t+1} + \gamma V(S_{t+1})$ and the **TD error** is

$$\delta_t \;=\; R_{t+1} + \gamma V(S_{t+1}) - V(S_t).$$

The TD(0) update with step size $\alpha \in (0, 1]$:

$$V(S_t) \leftarrow V(S_t) + \alpha\, \delta_t.$$

Only the current state is updated per step (tabular case). If $\gamma = 0$, the target reduces to R_{t+1} (pure immediate reward).

Convergence of Tabular On-Policy TD(0)

TD(0) prediction converges to v_π if (i) the policy is fixed and on-policy, (ii) all states are visited infinitely often, and (iii) step sizes satisfy the Robbins–Monro conditions per state:

$$\sum_t \alpha_t(s) = \infty, \qquad \sum_t \alpha_t(s)^2 < \infty.$$

Off-policy TD and/or function approximation can diverge.

TD(0) Update Calculation

$V(S_t) = 2$, $R_{t+1} = -1$, $\gamma = 0.9$, $V(S_{t+1}) = 3$:

$$\delta_t = -1 + 0.9 \cdot 3 - 2 = -0.3.$$

With $\alpha = 0.1$: $V(S_t) \leftarrow 2 + 0.1(-0.3) = 1.97$.

Bias–Variance Trade-off

TD targets $(R_{t+1} + \gamma V(S_{t+1}))$ are typically **lower variance** than MC returns — they use one reward plus a single estimate rather than a long sum of random rewards. But they are **more biased** early on, because $V(S_{t+1})$ is an imperfect approximation. MC targets are unbiased w.r.t. the true return but can have high variance, especially when episodes are long or rewards are noisy.

TD vs DP (Exam Nuance)

DP uses *expected* backups: it sums over all possible successors using the model. TD uses *sample* backups: it observes one successor. In a deterministic environment the sample equals the expectation, so a single TD step is equivalent to one DP backup. The step size α affects convergence speed, not the fixed point (for tabular on-policy prediction).

5.2 TD control

For control, TD learns **action values** $Q(s, a)$. The generic TD control update is:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big(R_{t+1} + \gamma\,\mathrm{Target}(S_{t+1}) - Q(S_t, A_t)\Big).$$

The target differs between algorithms; that difference defines the key on-policy/off-policy distinction. For any policy π: $v_\pi(s) = \sum_a \pi(a \mid s)\, q_\pi(s, a)$; for deterministic π, $v_\pi(s) = q_\pi(s, \pi(s))$.

5.2.1 SARSA (on-policy)

SARSA

On-policy TD control: the bootstrap target uses the next action *actually taken* by the current policy.

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma\, Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)).$$

Mnemonic: $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$.

In practice SARSA runs with an ε-greedy policy to ensure exploration. Because SARSA is on-policy, the *same* policy both generates experience and defines the bootstrap target via $A_{t+1} \sim \pi(\cdot \mid S_{t+1})$. If S_{t+1} is terminal, set $Q(S_{t+1}, \cdot) = 0$.

5.2.2 Off-policy SARSA and importance sampling

Off-Policy SARSA

To learn about target policy π while behaving under μ, apply an IS correction with ratio $\rho_{t+1} = \pi(A_{t+1} \mid S_{t+1})/\mu(A_{t+1} \mid S_{t+1})$:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\, \rho_{t+1}(R_{t+1} + \gamma\, Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)).$$

Q-learning (below) is off-policy and does *not* require IS, because its target uses $\max_a Q(S_{t+1}, a)$ — which is policy-agnostic.

5.2.3 Expected SARSA

Expected SARSA

Replace the sampled $Q(S_{t+1}, A_{t+1})$ by its expectation under π:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big(R_{t+1} + \gamma \sum_a \pi(a \mid S_{t+1}) Q(S_{t+1}, a) - Q(S_t, A_t)\Big).$$

Lower variance than SARSA, since we average over the random next action instead of sampling it.

Expected SARSA vs. Q-Learning

If the target policy is greedy/deterministic, the expectation collapses to a max: $\sum_a \pi(a \mid s')Q(s', a) = \max_a Q(s', a)$. Thus Expected SARSA with a greedy target policy *reduces to Q-learning*.

5.2.4 Q-learning (off-policy)

Q-Learning

Off-policy TD control: the target uses the *greedy* action at the next state, independent of the behavior policy:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)).$$

Under standard conditions (all (s, a) updated infinitely often, Robbins–Monro step sizes), Q converges to q_* w.p. 1.

The **behavior policy** μ generates data and should explore (e.g. ε-greedy). The **target policy** $\pi(s) = \arg\max_a Q(s, a)$ is the greedy policy being optimized. Although Q-learning is off-policy, no IS correction is needed: given (S_t, A_t), the next state is sampled from the environment dynamics, not from μ or π. The off-policy nature enters only through the max in the backup.

Maximization Bias

Using $\max_a Q(S_{t+1}, a)$ tends to *overestimate* action values in stochastic settings: the maximum over noisy estimates is systematically biased upward. **Double Q-learning** addresses this by maintaining two estimates Q_1, Q_2. For each transition, randomly choose

which to update: the updating network selects the action ($\arg\max_a Q_1$), but the other network evaluates it ($Q_2(s', a^*)$), decorrelating selection from evaluation.

Double Q-Learning

On each step, flip a fair coin.

If heads: $a^* = \arg\max_a Q_1(S_{t+1}, a), \quad Q_1(S_t, A_t) \leftarrow Q_1(S_t, A_t) + \alpha\big(R_{t+1} + \gamma\, Q_2(S_{t+1}, a^*) - Q_1(S_t, A_t)\big).$

If tails: symmetric with roles of Q_1 and Q_2 swapped. The random split decorrelates the two estimates and reduces maximization bias.

On-Policy vs. Off-Policy in One Line

$$\text{SARSA:} \quad \text{target uses } A_{t+1} \sim \pi(\cdot \mid S_{t+1}) \quad \text{(on-policy)}.$$

$$\text{Q-learning:} \quad \text{target uses } \arg\max_a Q(S_{t+1}, a) \quad \text{(off-policy)}.$$

5.3 n-step TD and the TD–MC spectrum

n-step TD interpolates between TD(0) ($n = 1$) and MC ($n = T - t$):

n-Step Return

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n}).$$

Then $V(S_t) \leftarrow V(S_t) + \alpha\big(G_t^{(n)} - V(S_t)\big)$. Special cases: $n = 1$ gives TD(0); $n = T - t$ removes the bootstrap term and recovers MC.

n-step TD can update before episode end: once n rewards have been observed, the partial trajectory suffices. If n exceeds the remaining episode length, the target becomes the full return (MC), not DP.

5.4 Eligibility traces and TD(λ)

Rather than committing to a single n, TD(λ) mixes all n-step returns with geometrically decaying weights.

λ-Return (Forward View)

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{T-t-1} \lambda^{n-1} G_t^{(n)} + \lambda^{T-t-1} G_t.$$

Special cases: $\lambda = 0$ gives TD(0); $\lambda \to 1$ approaches MC.

The forward view requires knowing the full future, making it online-unfriendly. Eligibility traces provide an equivalent *backward view*:

Eligibility Traces (Backward View)

For state values, maintain traces $e_t(s)$:

$$e_t(s) = \gamma\lambda\, e_{t-1}(s) + \mathbf{1}\{S_t = s\}.$$

Update all states proportionally to their trace:

$$V(s) \leftarrow V(s) + \alpha\, \delta_t\, e_t(s) \quad \text{for all } s.$$

For action values: $e_t(s,a) = \gamma\lambda\, e_{t-1}(s,a) + \mathbf{1}\{S_t = s, A_t = a\}$, with $Q(s,a) \leftarrow Q(s,a) + \alpha\, \delta_t\, e_t(s,a)$. The TD error δ_t depends on whether the underlying algorithm is SARSA, Expected SARSA, or Q-learning.

Batch TD vs. Batch MC

In the tabular case, batch TD(0) converges to the value function of the **maximum-likelihood Markov model** implied by the data (the DP solution on empirical transition dynamics). Batch MC converges to the solution minimizing mean-squared error on the observed returns. They generally converge to *different* solutions.

With **function approximation and bootstrapping** (semi-gradient TD, TD(λ), etc.), convergence is *not* guaranteed in general.

5.5 Exam questions

Exam: TD vs. MC

Q. What is an advantage of TD learning over Monte Carlo methods?

A. TD updates **online** after each step — no need to wait for episode end. This often yields faster learning in practice and works for continuing (non-episodic) tasks.

Exam: SARSA On-Policy

Q. "SARSA requires choosing a second action in the second state with the same policy as the first." True or false?

A. True. SARSA is on-policy: it needs $A_{t+1} \sim \pi(\cdot \mid S_{t+1})$ to form the target $Q(S_{t+1}, A_{t+1})$.

Exam: TD(λ)

Q. "TD(λ) computes a weighted average n-step TD backup target." True or false?

A. True. The λ-return is a geometric mixture of n-step returns (forward view), implemented efficiently via eligibility traces (backward view).

Common Exam Pitfalls — TD

- **Bootstrapping:** TD uses $V(S_{t+1})$ or $Q(S_{t+1}, \cdot)$ inside the target; MC does not.
- **SARSA vs. Q-learning:** SARSA uses the *actually taken* A_{t+1} (on-policy); Q-

learning uses $\max_a$ (off-policy).

- Expected SARSA takes an expectation *under the policy* at S_{t+1}; with a greedy target policy it reduces to Q-learning.
- Endpoints: $\lambda = 0 \Rightarrow$ TD(0); $\lambda \to 1 \Rightarrow$ MC.
- Eligibility traces **decay** by $\gamma\lambda$ and assign credit backward to recently visited states / state–action pairs.

6

Policy Gradient Methods

Value-based methods (Q-learning, SARSA) represent the policy implicitly via a value function: act greedily. **Policy gradient methods** instead parameterize the policy directly and optimize it by gradient ascent on an objective $J(\theta)$. This is natural for continuous action spaces (where $\arg\max_a Q(s,a)$ is hard), for stochastic policies, and wherever a differentiable policy representation is available.

6.1 The policy gradient objective

Parameterized Policy and Objective

A **policy gradient (PG) method** optimizes $\pi_\theta(a \mid s)$ directly by gradient ascent: $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$. A common objective is the expected discounted return from a start-state distribution:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_{t=0}^{T-1} \gamma^t R_{t+1}\right],$$

where $\tau = (S_0, A_0, R_1, S_1, \dots)$ is a trajectory induced by π_θ.

6.2 REINFORCE

The key mathematical tool is the *log-derivative trick*:

Score Function / Log-Derivative Identity

$$\nabla_\theta \pi_\theta(a \mid s) = \pi_\theta(a \mid s)\, \nabla_\theta \log \pi_\theta(a \mid s).$$

Applying this inside the expectation over trajectories yields the policy gradient theorem:

$$\nabla_\theta J(\theta) \propto \mathbb{E}_{\pi_\theta}\left[\sum_t \nabla_\theta \log \pi_\theta(A_t \mid S_t)\, q_{\pi_\theta}(S_t, A_t)\right].$$

No knowledge of the environment dynamics $p(s', r \mid s, a)$ is needed.

Replacing q_{π_θ} with the observed Monte Carlo return G_t gives the REINFORCE algorithm:

REINFORCE (MC Policy Gradient)

After completing an episode, for each time step t:

$$\theta \leftarrow \theta + \alpha\, G_t\, \nabla_\theta \log \pi_\theta(A_t \mid S_t).$$

Increase the probability of actions that led to high return; decrease it for actions with low return.

REINFORCE is an **unbiased** estimator of $\nabla_\theta J(\theta)$ but can have **high variance**, especially in long episodes. It is also **on-policy**: trajectories must come from the current π_θ.

Course/Exam Convention

In this course, REINFORCE refers to the **episodic Monte Carlo** variant that uses full episode returns G_t. The claim "REINFORCE is effective for both episodic and non-episodic MDPs" is therefore **false** under this convention: continuing tasks require additional machinery (average-reward formulations or actor–critic with TD critics).

6.3 Baselines and variance reduction

Baseline Trick

For any function $b(s)$ that does not depend on the action A_t:

$$\mathbb{E}_{\pi_\theta}[b(S_t)\, \nabla_\theta \log \pi_\theta(A_t \mid S_t)] = 0.$$

Therefore G_t can be replaced by $G_t - b(S_t)$ without changing the expected gradient, but typically reducing variance (the baseline shifts all action probabilities equally in expectation, canceling out):

$$\theta \leftarrow \theta + \alpha\, (G_t - b(S_t))\, \nabla_\theta \log \pi_\theta(A_t \mid S_t).$$

Common baselines: $b(s) = \bar{G}$ (running mean return), or $b(s) = V_w(s)$ (a learned state-value function).

Advantage Function

When $b(s) = v_\pi(s)$, the quantity $G_t - b(S_t)$ estimates the **advantage**:

$$A_\pi(s, a) = q_\pi(s, a) - v_\pi(s).$$

The advantage answers "how much better than average was action a in state s?" — a cleaner and lower-variance learning signal.

If $b(s) = V_w(s)$ is itself learned, the setup is called **actor + critic**. It is *not* a TD method unless the critic is trained with TD targets (e.g. using δ_t).

6.4 Actor–critic

Actor–critic methods combine an **actor** (the parameterized policy π_θ) with a **critic** (a value estimator V_w or Q_w). The critic provides a TD-style learning signal to update the actor; the

actor update only needs the scalar δ_t or $\hat{A}_t$ from the critic — it does not need $\nabla_w V_w$.

The one-step TD error serves as an advantage estimate:

Actor–Critic Updates

Critic (TD(0) on V_w):

$$\delta_t = R_{t+1} + \gamma V_w(S_{t+1}) - V_w(S_t), \qquad w \leftarrow w + \beta\, \delta_t\, \nabla_w V_w(S_t).$$

Actor:

$$\theta \leftarrow \theta + \alpha\, \delta_t\, \nabla_\theta \log \pi_\theta(A_t \mid S_t).$$

REINFORCE vs. Actor–Critic

Compared to REINFORCE, actor–critic can update **online** (no waiting for episode end) and typically has lower variance. The cost is **bias** introduced by bootstrapping from the approximate critic. This is the classic bias–variance trade-off that runs through all of RL.

6.4.1 PPO (brief)

Proximal Policy Optimization (PPO)

PPO stabilizes policy updates via a **clipped surrogate** objective. With probability ratio $r_t(\theta) = \pi_\theta(A_t \mid S_t)/\pi_{\theta_{\text{old}}}(A_t \mid S_t)$:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}\Big[\min\big(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta),\, 1-\epsilon,\, 1+\epsilon)\hat{A}_t\big)\Big].$$

Clipping prevents any single update from moving the policy too far from $\pi_{\theta_{\text{old}}}$, stabilizing training without a hard constraint.

6.5 Exam questions

Exam: Baseline Unbiasedness

Q. Why can we subtract a baseline $b(S_t)$ from the return in REINFORCE without biasing the gradient?

A. Because $\mathbb{E}[b(S_t)\nabla_\theta \log \pi_\theta(A_t \mid S_t)] = 0$ under $A_t \sim \pi_\theta(\cdot \mid S_t)$, so the baseline term vanishes in expectation and does not affect $\mathbb{E}[\nabla_\theta J(\theta)]$.

Exam: REINFORCE Variance

Q. "REINFORCE is unbiased but often high variance." True or false?

A. True. It uses Monte Carlo returns; variance can be large without baselines or critics.

Exam: Online Actor–Critic

Q. "Actor–critic methods can update before an episode ends." True or false?

A. True. The critic provides a TD-style signal that bootstraps from the next state, enabling online updates after every step.

Common Exam Pitfalls — Policy Gradients

- PG is **gradient ascent** on $J(\theta)$, not descent.
- Updates use $\nabla_\theta \log \pi_\theta(A_t \mid S_t)$, not $\nabla_\theta \pi_\theta$ directly.
- The baseline must not depend on the chosen action; otherwise it introduces bias.
- Actor–critic reduces variance but introduces bias (bootstrapping); REINFORCE is unbiased but high variance.
- G_t is the MC return; δ_t is the TD error / one-step advantage estimate — do not confuse them.
- With expert state–action pairs (s, a), a policy can be pre-trained via supervised learning ($\max_\theta \log \pi_\theta(a \mid s)$) before RL fine-tuning.